

AI Agent 学习笔记（第一天）

身份背景：前端开发工程师（7 年经验），从 0 开始学习 AI Agent。

一、什么是 Agent?

最初理解

Agent 就像软件开发中的中间件，用于承接 AI 需要做的事情，把人的工作流包装起来交给 AI 完成。

修正后的理解：

Agent = LLM + Runtime + Context + Tools/Skills

其中：

- **LLM**：负责推理、规划、决策。
 - **Runtime**：负责驱动整个执行循环。
 - **Context**：当前任务的上下文和状态。
 - **Tools/Skills**：Agent 可以调用的能力。
-

二、Agent 的核心循环

Agent 本质上不是一次调用 LLM。

而是不断循环：

Goal

↓

LLM 推理 (Reason)

↓

决定是否调用 Tool

↓

执行 Tool (Act)

↓

获得结果 (Observe)

↓

更新 Context

↓

继续推理

↓

直到完成

可以理解成：

```
while (!finished) {  
    response = llm(context)  
  
    if (response.needTool) {  
        result = tool()  
        context.push(result)  
        continue  
    }  
  
    break  
}
```

真正重要的是：

LLM 根据 Observation 动态决定下一步，而不是程序员写死流程。

三、Tool 和 Skill 的区别

Tool

粒度比较小。

例如：

- readFile
- writeFile
- Browser
- Shell
- Git
- SQL

一个 Tool 基本对应一个能力。

Skill

Skill 一般由多个 Tool 组成。

例如：

修复 React Bug：

```
readFile()
```

↓

```
grep()
```

↓

```
terminal()
```

↓

```
writeFile()
```

↓

```
npm test()
```

Skill 更接近业务能力。

四、企业为什么要做自己的 Agent?

不是为了超过 Claude Code。

而是：

Claude Code 不知道企业内部的信息。

例如：

- CRM
- ERP
- OA
- Jira
- GitLab
- Jenkins

- 内部 Wiki
- 内部组件库
- 公司规范

企业真正做的是：

LLM

↓

Company Runtime

↓

Company Tools

↓

Company Knowledge

↓

Company Workflow

也就是：

让 Agent 成为最懂公司的 AI。

五、Agent 开发主要开发什么？

真正开发的不是 LLM。

而是：

Tool

例如：

- 查询订单
- 查询 CRM
- 查询 Jira
- Git
- Browser
- 发邮件

很多企业 Agent，80% 的代码都是 Tool。

Context

Context 包括：

- 当前目标
- 历史聊天
- Tool 返回结果
- Memory
- RAG
- Prompt

LLM 每次收到的都是 Context。

Runtime

Runtime 负责：

- 调用 LLM
 - Tool 调度
 - Retry
 - Timeout
 - Human Approval
 - Streaming
 - Error Handle
 - Memory
-

六、Agent Framework

Agent Framework 的作用：

就是帮开发者写 Runtime。

目前主流：

- OpenAI Agents SDK
- LangGraph
- Mastra
- CrewAI
- AutoGen
- LlamaIndex

其中：

对于 TypeScript / Node.js 开发者：

Mastra 和 OpenAI Agents SDK 更容易上手。

七、Agent Framework 最终产出的是什么？

不是 ChatGPT 网页。

而是：

一个服务。

例如：

```
React
↓
POST /chat
↓
Agent Runtime
↓
LLM
↓
返回 Stream
```

真正的聊天页面：

需要自己开发。

八、AI 产品整体架构

```
React / Next.js
↓
HTTP / SSE
↓
Agent Server
↓
```

LLM

↓

Tools

其中：

React：

负责 UI。

Agent：

负责 AI。

九、AI 前端主要负责什么？

包括：

- Chat UI
- 会话管理
- 文件上传
- Streaming
- Markdown
- Tool 状态
- Human Approval
- 多 Agent 展示
- Workflow 展示

AI 前端：

更像：

展示 AI 的工作过程。

十、Conversation 和 Context 的区别

Conversation：

数据库里的持久化数据。

例如：

conversationId



数据库



历史 Message

Context:

每次请求时重新构建。

例如:

System Prompt

+

Conversation Memory

+

User Memory

+

Knowledge(RAG)

+

当前消息

然后:

一起发送给 LLM。

LLM:

不知道 conversationId。

它只知道 Context。

十一、Memory

Memory 一般分三层：

Conversation Memory

当前会话。

例如：

最近二十条聊天。

User Memory

跨会话。

例如：

用户偏好：

- TypeScript
 - React
 - 英文水平
-

Knowledge

企业知识。

例如：

- Wiki
 - API
 - 设计规范
 - 组件库
-

十二、会话创建流程

通常：

用户点击：

New Chat

↓

React:

POST /conversation

↓

后端:

生成 conversationId

↓

React:

进入:

/chat/{conversationId}

以后:

所有消息:

都带:

conversationId。

另一种方案:

第一次发送消息时:

conversationId 为空。

Agent 自动创建。

十三、Agent 后端负责什么？

负责:

- Runtime
- Tool
- Memory
- Context
- LLM
- Event Stream

不会负责：

React 页面。

十四、前端真正难的地方

不是文件上传。

真正难的是：

状态管理。

例如：

Agent：

可能经历：

Thinking

↓

Planning

↓

Tool Running

↓

Need Approval

↓

Continue

↓

Done

这些状态：

都需要 React 去维护。

十五、Streaming 为什么比普通 SSE 更复杂？

普通 SSE：

只有：

message

↓

message

↓

message

Agent：

返回的是：

事件流。

例如：

thinking

↓

tool_start

↓

tool_finish

↓

message_delta

↓

approval_required

↓

finish

所以：

前端需要根据 Event 更新不同 UI。

十六、Agent 前后端职责划分

Agent 后端

负责：

- Tool 执行
 - Runtime
 - Context
 - LLM
 - Event Stream
-

React 前端

负责：

- 接收 Event
- 更新 Store
- Streaming
- Tool UI
- Approval UI
- Chat UI

前端并不知道 Tool 内部如何执行。

它只负责展示 Tool 的状态。

十七、当前学习结论

目前最大的认知收获：

1. Agent 不只是 Tool Calling，而是 Runtime 驱动下的推理循环。
2. 企业开发 Agent 的重点不是训练模型，而是连接企业内部系统和知识。
3. Agent Framework（OpenAI Agents SDK、Mastra 等）主要帮助开发 Runtime。
4. Conversation 是数据库里的数据；Context 是每次请求动态构建后发送给 LLM 的上下文。
5. AI 前端的核心不只是聊天框，而是展示 Agent 的执行过程、管理复杂状态和处理事件流。
6. Tool 的执行发生在后端；前端负责接收事件并展示 Tool 的执行状态。
7. 对于有前端和 Node.js 经验的工程师来说，学习 Agent 更多是在学习 Runtime、Context、Tool，而不是训练大模型。